

Laboratory 03

Aperiodic Servers & Producer–Consumer Pipeline

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	3–4 hours
CLO mapping	CLO 2 & CLO 3

1. Objectives

By completing this lab you will be able to:

1. Implement a **deferred-interrupt handler** that moves ISR work into a task context.
2. Implement a **Polling Server** pattern using a FreeRTOS software timer and a request queue.
3. Measure **aperiodic response time** under a periodic background load and compare Polling Server vs. direct task notification.
4. Build a **two-producer, one-consumer** queue pipeline and observe queue saturation.
5. Use an **event group** to coordinate a multi-condition synchronisation scenario.

2. Prerequisites

- ☐ Lab 02 complete — you can build, flash, and observe a FreeRTOS application.
- ☐ ARM GNU Toolchain and pyocd working.
- ☐ FRDM-MCXN236 board connected via MCU-LINK USB.
- ☐ SEGGER SystemView installed (needed for Part D).

3. Background

The course Task Model assumes **periodic** tasks with known execution times. Real systems also receive **aperiodic** events (button presses, UART interrupts, sensor alerts) that must be handled without jeopardising periodic task deadlines.

Two strategies are in scope for this lab:

Strategy	Mechanism	Response time	Periodic impact
Deferred interrupt	ISR gives semaphore → task unblocks immediately	Very low	Depends on handler task priority
Polling Server	Software timer wakes a service task on a fixed budget and period	Up to one server period	Bounded — server is treated as a periodic task

4. Project Structure

```
lab03_aperiodic_server/
  Makefile
  src/
    main.c           <- creates all tasks and kernel objects
    periodic_tasks.c/.h <- tau1 (100 ms), tau2 (200 ms) background load
    aperiodic_handler.c/.h <- deferred ISR task + Polling Server task
```

```
producer_consumer.c/.h <- two-producer / one-consumer pipeline (Part C)
FreeRTOSConfig.h
```

Note

The FreeRTOS kernel and all hardware drivers come from the NXP MCUXpresso SDK — no separate kernel clone is needed. Hardware initialisation (clocks, pin-mux, LPUART4) is handled by `BOARD_InitHardware()` from the SDK board-support package. Use `PRINTF` from `fsl_debug_console.h` for all console output.

4.1. Task and Kernel Object Map

Object	Type	Notes
vPeriodic1Task	Task prio 3, T=100 ms	Background periodic load
vPeriodic2Task	Task prio 2, T=200 ms	Background periodic load
vAperiodicHandlerTask	Task prio 4	Deferred ISR handler (Part A)
vPollingServerTask	Task prio 3	Budget-limited service task (Part B)
xAperiodicSem	Binary semaphore	ISR → handler signal
xRequestQueue	Queue depth 4	Polling Server request queue
vSensor1Task	Task prio 3, T=100 ms	Producer 1 (Part C)
vSensor2Task	Task prio 2, T=200 ms	Producer 2 (Part C)
vLoggerTask	Task prio 1	Consumer (Part C)
xPipelineQueue	Queue depth 10	Sensor → Logger (Part C)
xPipelineEvents	Event group	DATA_READY + ALARM bits (Part C)

5. Part A — Deferred Interrupt Handler

5.1. Step 1 — Wire the Button ISR

The FRDM-MCXN236 has a user button on GPIO P0_23. The ISR is already registered in `main.c`; your task is to implement the handler.

In `aperiodic_handler.c`, implement:

```
1  /* ISR -- keep it minimal: signal and return */
2  void GPIO0_IRQHandler(void)
3  {
4      BaseType_t xHigherPriorityTaskWoken = pdFALSE;
5      GPIO_ClearPinsInterruptFlags(GPIO0, 1u << 23);
6      xSemaphoreGiveFromISR(xAperiodicSem, &xHigherPriorityTaskWoken);
7      portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
8  }
9
10 /* Handler task -- runs at prio 4, higher than all periodic tasks */
11 void vAperiodicHandlerTask(void *pv)
12 {
13     uint32_t count = 0;
```

```

14     for (;;) {
15         xSemaphoreTake(xAperiodicSem, portMAX_DELAY);
16         count++;
17         PRINTF("[ASYNC] button press #%u -- handled in task context
18                 t=%u ms\r\n",
19                 count, (unsigned)xTaskGetTickCount());
20     }

```

5.2. Step 2 — Build and Test

```

cd labs/lab03_aperiodic_server
make
make flash

```

Press the user button several times and observe the terminal. Each press should appear as an [ASYNC] line between the [P1] and [P2] periodic lines.

✓ Checkpoint A

Terminal showing at least 3 button presses interleaved with periodic output, with no missed presses.

? Question A1

Press the button rapidly 5 times in under 100 ms. Do any presses get lost? Why or why not? (*Hint: xAperiodicSem is a binary semaphore. What happens to the second give if the first has not been taken yet?*)

? Question A2

The handler task runs at priority 4, above all periodic tasks. What is the cost of this choice? When would you set the handler priority *below* the periodic tasks?

6. Part B — Polling Server

6.1. Step 1 — Implement the Server

A Polling Server is a periodic task with a fixed budget B_s . It checks a request queue and drains up to B_s ms of work per activation.

In `aperiodic_handler.c`:

```

1  #define POLLING_SERVER_BUDGET_MS    10    /* max service per period */
2  #define POLLING_SERVER_PERIOD_MS    50    /* Ts */
3
4  void vPollingServerTask(void *pv)
5  {
6      TickType_t xLastWake = xTaskGetTickCount();
7      for (;;) {
8          vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(
9              POLLING_SERVER_PERIOD_MS));
10
11         TickType_t budget_start = xTaskGetTickCount();
12         AperiodicRequest_t req;
13
14         while ((xTaskGetTickCount() - budget_start) <
15             pdMS_TO_TICKS(POLLING_SERVER_BUDGET_MS))
16             {

```

```

16         if (xQueueReceive(xRequestQueue, &req, 0) != pdTRUE)
17             break;
18         PRINTF("[PS] served request type=%d arrived=%u ms
19             served=%u ms\r\n",
20                 req.type, req.timestamp, (unsigned)
21                 xTaskGetTickCount());
22     }

```

Note

The Polling Server is parameterised as a periodic task with $C_i = B_s$, $T_i = T_s$ for schedulability analysis. Add it to the utilisation calculation — does the total U stay below $\ln 2$?

6.2. Step 2 — Submit Requests from an ISR

Wire the button ISR to enqueue a request instead of giving a semaphore (switch `main.c` to `#define LAB_PART POLLING_SERVER`):

```

1 void GPIO0_IRQHandler(void)
2 {
3     BaseType_t xHPTW = pdFALSE;
4     GPIO_ClearPinsInterruptFlags(GPIO0, 1u << 23);
5     AperiodicRequest_t req = { .type = 1,
6                               .timestamp = xTaskGetTickCountFromISR
7                                       () };
8     xQueueSendFromISR(xRequestQueue, &req, &xHPTW);
9     portYIELD_FROM_ISR(xHPTW);

```

6.3. Step 3 — Observe and Measure

Press the button and note the time between the press (timestamp logged in the ISR) and the `[PS] served` line.

Checkpoint B

Terminal output with at least 5 requests served, annotated with arrival time and service time.

Question B1

What is the worst-case aperiodic response time of the Polling Server in terms of T_s ? With $T_s = 50$ ms, what is the maximum expected wait? Does your measured data agree?

Question B2

You pressed the button 8 times quickly. The queue depth is 4. What happened to the other 4 requests? What would change if you used `xQueueSendFromISR` with `portMAX_DELAY`?

Question B3

Compare the Polling Server with the direct deferred handler from Part A. Fill in this table from your observations:

Metric	Deferred handler	Polling Server
Response time (typical)		
Response time (worst case)		
Effect on periodic tasks at peak load		
Analysis complexity		

7. Part C — Two-Producer, One-Consumer Pipeline

7.1. Step 1 — Build the Pipeline

Implement in `producer_consumer.c`:

```

1 typedef struct {
2     uint8_t  sensor_id;
3     int16_t  value;
4     uint32_t timestamp;
5 } SensorReading_t;
6
7 void vSensor1Task(void *pv)    /* T=100 ms, prio 3 */
8 {
9     static int16_t raw = 0;
10    TickType_t xLastWake = xTaskGetTickCount();
11    for (;;) {
12        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(100));
13        raw = (raw + 7) % 1000;
14        SensorReading_t r = { .sensor_id = 1, .value = raw,
15                             .timestamp = xTaskGetTickCount() };
16        if (xQueueSend(xPipelineQueue, &r, 0) != pdTRUE)
17            PRINTF("[S1] queue full -- dropped!\r\n");
18    }
19 }
20
21 void vSensor2Task(void *pv)    /* T=200 ms, prio 2 */
22 {
23     static int16_t raw = 500;
24     TickType_t xLastWake = xTaskGetTickCount();
25     for (;;) {
26         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(200));
27         raw = (raw + 13) % 1000;
28         SensorReading_t r = { .sensor_id = 2, .value = raw,
29                              .timestamp = xTaskGetTickCount() };
30         xQueueSend(xPipelineQueue, &r, 0);
31     }
32 }
33
34 void vLoggerTask(void *pv)     /* prio 1 -- lowest */
35 {
36     static uint32_t count = 0;
37     SensorReading_t r;
38     for (;;) {
39         xQueueReceive(xPipelineQueue, &r, portMAX_DELAY);
40         count++;
41         PRINTF("[LOG] %u sensor=%d val=%d t=%u ms\r\n",
42               count, r.sensor_id, r.value, r.timestamp);

```

```

43     if (r.value > 800)
44         xEventGroupSetBits(xPipelineEvents, BIT_ALARM);
45     if ((count % 10) == 0)
46         xEventGroupSetBits(xPipelineEvents, BIT_DATA_READY);
47 }
48 }

```

7.2. Step 2 — Add Event Group Coordination

A fourth task waits for BIT_DATA_READY (every 10 items processed) OR BIT_ALARM (value > 800):

```

1  #define BIT_DATA_READY    (1 << 0)
2  #define BIT_ALARM        (1 << 1)
3
4  void vMonitorTask(void *pv)    /* prio 2 */
5  {
6      for (;;) {
7          EventBits_t bits = xEventGroupWaitBits(
8              xPipelineEvents,
9              BIT_DATA_READY | BIT_ALARM,
10             pdTRUE,    /* clear on exit */
11             pdFALSE,   /* OR -- either bit suffices */
12             portMAX_DELAY);
13         if (bits & BIT_DATA_READY)
14             PRINTF("[MON] 10-item batch complete t=%u ms\r\n",
15                 (unsigned)xTaskGetTickCount());
16         if (bits & BIT_ALARM)
17             PRINTF("[MON] ALARM: value exceeded threshold t=%u ms\r\n",
18                 (unsigned)xTaskGetTickCount());
19     }
20 }

```

Set BIT_ALARM inside vLoggerTask when r.value > 800.

✓ Checkpoint C

Terminal showing both [MON] 10-item batch and [MON] ALARM lines with correct timing.

? Question C1

Why is the event group AND (pdTRUE) vs. OR (pdFALSE) distinction important here? What would happen if you used AND for this monitor task?

? Question C2

Use uxQueueMessagesWaiting(xPipelineQueue) inside vLoggerTask and print it every 10 items. Does the queue ever have more than 1 item waiting? Explain why or why not at current production vs. consumption rates.

? Question C3

Artificially slow down vLoggerTask with vTaskDelay(pdMS_TO_TICKS(150)) after each print. Watch [S1] queue full messages appear. What is the queue high-water mark just before the first drop? How does this inform the minimum required queue depth for a 2-second burst?

8. Part D — SystemView Trace

Rebuild without the slowing `vTaskDelay` from Question C3. Capture a SystemView trace of the full system (all tasks running).

1. Open SystemView → Start Recording.
2. Run for at least 5 seconds.
3. Stop recording and export the trace.

In the trace identify and annotate:

- A context switch from `vLoggerTask` to `vSensor1Task` triggered by timer expiry.
- A `xQueueSend` call inside `vSensor1Task` that directly unblocks `vLoggerTask`.
- The [MON] task sleeping until the event group fires.

✓ Checkpoint D

Annotated screenshot of the SystemView timeline.

9. Deliverables

#	Item	Details
1	Part A terminal capture	3+ button presses interleaved with periodic output
2	Part B comparison table	Question B3 with measured values
3	Part C terminal capture	Both event group triggers visible
4	Part D SystemView screenshot	Annotated timeline
5	Written answers	Questions A1–A2, B1–B3, C1–C3
6	<code>FreeRTOSConfig.h</code>	Utilisation calculation comment including Polling Server

10. Key Concepts Introduced

Concept	Where you saw it
Deferred interrupt handler pattern	Part A
Binary semaphore for ISR→task signalling	Part A
Polling Server budget and period	Part B
Aperiodic response time under periodic load	Parts A & B
Multi-producer queue pipeline	Part C
Queue saturation and flow control	Part C
Event group AND vs. OR synchronisation	Part C

11. Reference

Resource	Relevant sections
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 6 (semaphores), Ch. 8 (event groups)
Buttazzo, <i>Hard Real-Time Computing Systems</i>	Ch. 5 (aperiodic task scheduling)
FreeRTOS docs	<code>xSemaphoreGiveFromISR</code> , <code>xQueueSendFromISR</code> , <code>xEventGroupSetBits</code> , <code>xEventGroupWaitBits</code>
